SOFTWARE ENGINEERING & ARCHITECTURE SPECIFICATION

Smart Emergency Response & Ambulance Dispatch System

Complete End-to-End System Manual, Modules Breakdown, Database Schema, REST API Directory & Algorithmic Dispatch Architecture

---

PLATFORM & FRAMEWORK

.NET 8 Minimal API & React Native / Expo

DATABASE ENGINE

Microsoft SQL Server / EF Core

SECURITY & GEOLOCATION

JWT RBAC & Haversine Geodesic Math

TARGET DOMAINS

Emergency Healthcare & Fleet Logistics

DOCUMENT STATUS

Complete Official Specification (v1.0)

AUTHOR / TEAM

Full Engineering Development Group

Document Table of Contents

1. **Section 1: Executive Summary & Problem Formulation**
2. **Section 2: High-Level System Architecture & Technology Stack**
3. **Section 3: Comprehensive Module-by-Module Specification**
    - 3.1 Module 1: Citizen / Patient Emergency Access Portal (Zero-Login Flow)
    - 3.2 Module 2: Hospital Command Center Portal (Fleet & Admissions)
    - 3.3 Module 3: Ambulance Driver Telemetry & Navigation Mobile Console
    - 3.4 Module 4: Central Dispatcher & Super-Admin Control Room
    - 3.5 Module 5: Operations Archive & Historical Trip Audit Module
    - 3.6 Module 6: Live Performance Analytics, Metrics & Reports Module
    - 3.7 Module 7: Core Geospatial & Dispatching Engine (Haversine Optimization)
    - 3.8 Module 8: Security, JWT Authentication & Role-Based Access Control (RBAC)
4. **Section 4: Database Design, Schema & Relational Dictionary**
5. **Section 5: Complete REST API Specification & Endpoint Directory**
6. **Section 6: Algorithmic Formulations & Mathematical Modeling**
7. **Section 7: Dispatch State Machine & Operational Lifecycle**
8. **Section 8: Deployment, Setup & Smoke Verification Guide**

1. Executive Summary & Problem Formulation

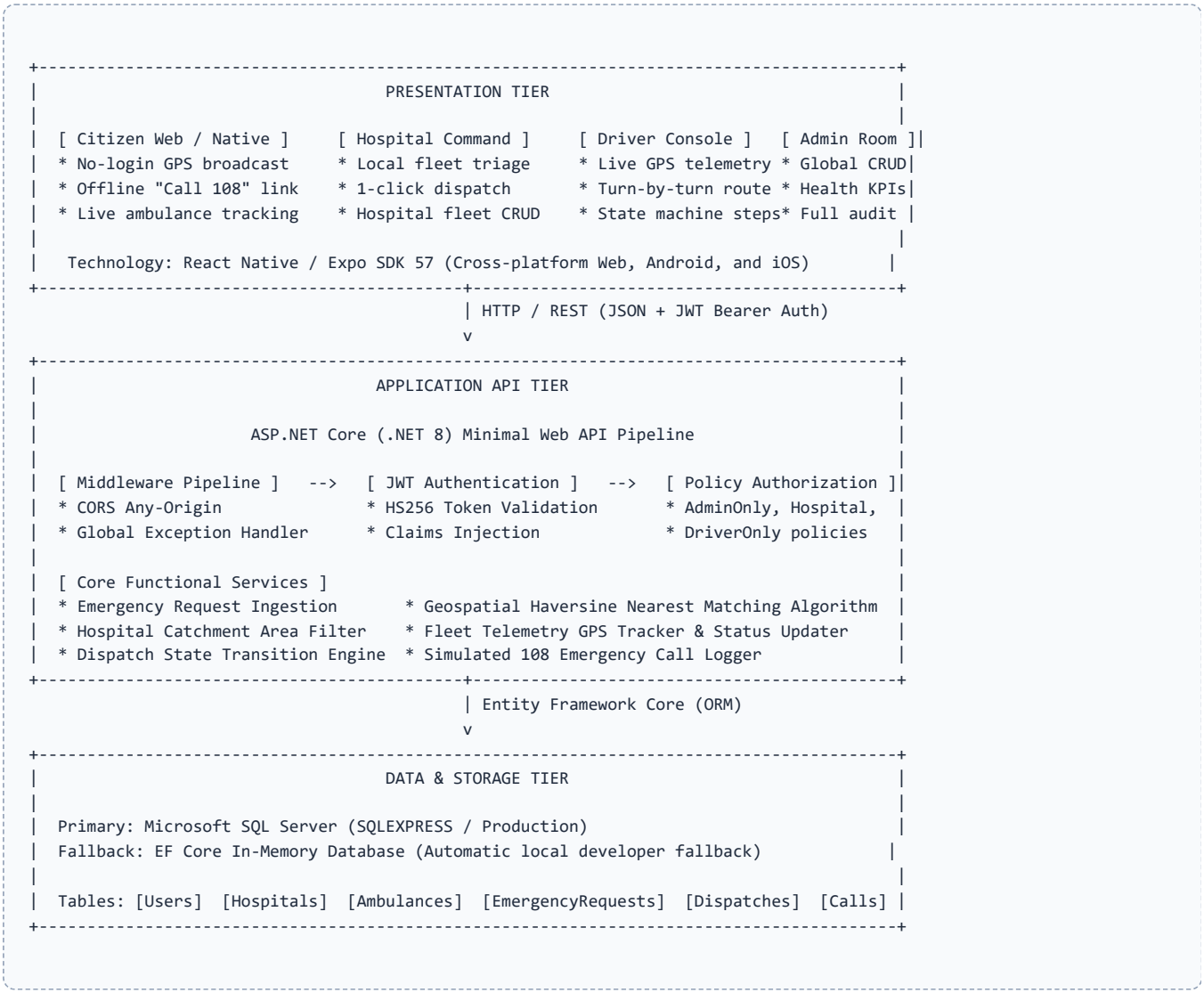
In emergency medicine, the concept of the **"Golden Hour"** dictates that rapid medical intervention within the first 60 minutes following severe trauma, cardiac arrest, or stroke drastically reduces mortality and long-term morbidity rates. Traditional emergency dispatching mechanisms face recurring friction points:

- **Caller Anxiety & Verbal Miscommunication:** Callers in distress struggle to provide accurate addresses, landmarks, or postal codes over cellular phone connections.
- **Manual Dispatch Bottlenecks:** Control centers manually contact individual hospitals and ambulance stations through telephone trees, delaying deployment.
- **Siloed Fleet Visibility:** Hospitals lack real-time visibility into mutual fleet availability and incoming trauma cases.
- **Driver Navigation Disconnect:** Ambulance drivers rely on oral phone instructions without exact GPS navigation coordinates to the patient.

> **The Core Solution:** The *Smart Emergency Response & Ambulance Dispatch System* creates a real-time, bi-directional emergency triage network. Citizens trigger zero-login GPS-pinned distress calls in under 5 seconds. The nearest participating hospital receives immediate catchment alerts and automatically assigns the closest available ambulance through the spherical Haversine geodesic algorithm. Drivers broadcast real-time telemetry updates as they transition from transit to patient pickup to hospital emergency room handover.

2. High-Level System Architecture & Technology Stack

The system follows a modern, decoupled N-Tier client-server micro-architecture engineered for fault tolerance, rapid startup, and cross-platform accessibility.



Technology Stack Matrix

LAYER	TECHNOLOGY	VERSION / SPECS	ARCHITECTURAL JUSTIFICATION
Mobile / Native	React Native / Expo SDK	SDK 51/57, CameraView, Location	Unified code base for native Android/iOS mobile emergency apps and QR-pairing.
Web Frontend	React 18 & Leaflet Maps	ES6+, HTML5 Geolocation, Leaflet CSS	Zero-lag desktop control room with interactive OpenStreetMap geospatial tracking.
Backend Framework	ASP.NET Core Minimal APIs	.NET 8.0 C#	Minimal overhead, sub-millisecond route handling, native asynchronous I/O (`async/await`).
Database Engine	Microsoft SQL Server / EF Core	SQL Server 2019+ / EF Core 8.0	Relational integrity, foreign key constraints, ACID compliance, and migration tracking.

LAYER	TECHNOLOGY	VERSION / SPECS	ARCHITECTURAL JUSTIFICATION
Authentication	JWT Bearer Tokens	System.IdentityModel.Tokens (HS256)	Stateless authorization with embedded claims (`Role`, `hospitalId`, `ambulanceId`).
Geodesic Engine	Spherical Haversine Algorithm	Pure C# Trigonometric Implementation	Instant geodesic distance calculations without external paid API rate limits or latency.

3. Comprehensive Module-by-Module Specification

The system is decomposed into 8 interconnected functional modules. Each module encapsulates dedicated business logic, user interface views, data models, and API contracts.

3.1 Module 1: Citizen / Patient Emergency Access Portal

Public / No Login

Component Files: `frontend/src/components/Emergency.js`, `frontend/App.js` (Native Mobile), `backend/Program.cs` (`/api/emergency`).

Purpose & Philosophy: When a medical catastrophe occurs, users cannot be hindered by registration screens, password prompts, or email verifications. This module provides a friction-free, instantaneous gateway to emergency dispatch.

Key Functional Capabilities

- **Single-Tap GPS Acquisition:** Queries device GPS sensors (via HTML5 Geolocation API on web or `expo-location` on native) to obtain precise Latitude and Longitude coordinates.
- **Offline "Call 108" Dial Link:** Built-in `tel:108` protocol trigger that directly opens the native phone dialer to reach national emergency services even when mobile internet data fails.
- **Triage Categorization:** Rapid radio selectors for Emergency Type (*Accident, Cardiac, Trauma, Respiratory, Maternal*) and Priority level (*Critical, Urgent, Normal*).
- **Live Ambulance Tracker:** Once submitted, polls the backend (every 5 seconds) to detect hospital acceptance, displaying vehicle license plate, driver phone, and live vehicle location on Leaflet map.

Data Inputs & Outputs

- **User Inputs:** Patient Name, Phone Number, Emergency Type, Priority, GPS Coordinates (auto-filled).
- **Output / Feedback:** Unique `RequestId`, status tracker badge (*Pending* -> *Accepted* -> *En Route* -> *Arrived*), ambulance contact details, and hospital name.
- **Native Feature:** Camera QR Code scanner (`CameraView`) to auto-pair mobile apps to the hospital local LAN server.

3.2 Module 2: Hospital Command Center Portal

Role: Hospital

Component Files: `frontend/src/components/HospitalView.js`, `backend/Program.cs` (`/api/hospitals`, `/api/emergency/available-for-hospital/{id}`, `/api/emergency/{id}/accept`).

Purpose & Philosophy: Empowers hospital staff to monitor incoming distress calls within their geographic catchment area and dispatch their private or affiliated ambulance fleet.

Key Functional Capabilities

1. **Hospital Authentication:** Secure passwordless token generation verifying registered hospital contact telephone number (claims injected: `Role: Hospital`, `hospitalId: X`).
2. **Geofenced Emergency Feed:** Backend automatically filters all active pending emergency requests. Using the Haversine formula, it only serves requests whose nearest hospital is *this* facility, preventing multi-hospital dispatch chaos.
3. **One-Click Dispatch Acceptance:** Clicking "Accept Emergency" triggers an automated backend routine:
   - Finds the nearest `Available` ambulance belonging exclusively to this hospital.
   - Creates a new record in `Dispatches` table linking the Request, Ambulance, and Hospital.
   - Transitions Request status to `Accepted by Hospital` and Ambulance status to `Assigned`.
4. **Hospital Fleet Management:** Add new ambulances (Vehicle Number, Driver Name, Driver Phone, Vehicle Type), view live fleet availability, and decommission units.
5. **Active Mission Tracking:** Inspect assigned runs, see patient pickup location on map, and prepare emergency room staff prior to patient arrival.

3.3 Module 3: Ambulance Driver Telemetry & Navigation Mobile Console

Role: Driver

Component Files: `frontend/src/components/DriverView.js`, `backend/Program.cs` (`/api/auth/driver-login`, `/api/ambulances/{id}/location`, `/api/dispatch/{id}`).

Purpose & Philosophy: Designed specifically for in-vehicle mounted tablets or driver mobile phones. Provides clear, high-contrast operational status and automated real-time GPS telemetry broadcasting.

Continuous GPS Telemetry

Mounts a background `navigator.geolocation.watchPosition` worker that triggers HTTP PUT requests to `/api/ambulances/{id}/location` every few seconds. This keeps the centralized system and patient tracking maps accurate within meters.

Lifecycle State Controller

Drivers interact with large, tactile buttons that progress the dispatch state: `Assigned` → `En Route` → `Arrived at Scene` (records ArrivalTime) → `Transporting` → `Completed` (records CompletionTime, releases ambulance back to "Available").

Driver Mission HUD: Displays patient emergency priority badge (Critical / Urgent / Normal), patient contact phone with direct click-to-call link, address/landmark notes, patient GPS coordinates on interactive Leaflet map, and assigned destination hospital name.

3.4 Module 4: Central Dispatcher & Super-Admin Control Room

Role: Admin

Component Files: `frontend/src/components/AdminDashboard.js`, `frontend/src/components/AdminLogin.js`, `backend/Program.cs`.

Purpose: Complete administrative oversight of the entire city or state emergency response infrastructure.

Administrative Capabilities:

- **Fleet Registry CRUD:** Full create, read, update, and delete access across all ambulances in the region, including manual status overrides (e.g., placing vehicle in "Maintenance").
- **Hospital Network Directory:** Register new healthcare facilities with address, contact numbers, and precise GPS lat/lng coordinates.
- **Manual Dispatch Override:** In high-casualty incidents or inter-hospital transfers, administrators can manually force a dispatch assigning any available ambulance to any pending request.
- **System Diagnostics & Data Dump:** Live overview of database entity counts via `/api/admin/summary` and full JSON snapshot export via `/api/debug/dump`.

3.5 Module 5: Operations Archive & Historical Trip Audit Module

Operational Audit

Component Files: `frontend/src/components/HistoryView.js`, `backend/Program.cs` (`/api/emergency`, `/api/dispatch`, `/api/calls`).

Key Audit Features:

- **Universal Search Engine:** Instant client-side text filtering indexing across Patient Name, Contact Phone, Incident Address, and unique Request ID.
- **Status Facet Filtering:** Filter archive by trip outcome: *All, Pending, Accepted by Hospital, Assigned, En Route, Completed*.
- **Chronological Turnaround Milestones:** Records exact timestamps for each phase of the emergency:
  - `RequestTime` (when the citizen hit submit)
  - `AssignedTime` (when hospital dispatched ambulance)
  - `ArrivalTime` (when ambulance arrived at patient location)
  - `CompletionTime` (when patient was safely admitted to hospital ER)

3.6 Module 6: Live Performance Analytics, Metrics & Reports Module

Executive Intelligence

Component Files: `frontend/src/components/ReportsView.js`, `backend/Program.cs` (`/api/admin/summary`).

Core KPI Cards

- **Total Request Volume:** Cumulative count of all emergency calls ingested since system inception.
- **Active Dispatches:** Currently engaged ambulances in transit or treating patients.
- **Mission Completion Rate:** Percentage formula: `(Completed Dispatches / Total Dispatches) * 100`.
- **Priority Alerts:** Cumulative total of High, Critical, and Urgent trauma requests.

Resource Health & Demand Mix

- **Demand Distribution:** Dynamic bar chart breakdowns indicating the proportion of Critical vs Urgent vs Normal medical distress.
- **Fleet Availability Gauge:** Live ratio of Available ambulances against total registered vehicles.
- **Hospital Network Status:** Count of actively accepting Emergency Departments versus offline facilities.
- **Telephony Records:** Audit of simulated telephone calls logged via `/api/call`.

3.7 Module 7: Core Geospatial & Dispatching Engine

Mathematical Engine

Component Files: `backend/Program.cs` (Haversine implementation, `/api/ambulances/nearest`, `/api/hospitals/nearest`).

The dispatching engine computes the great-circle geodesic distance between two points on the Earth's surface using latitude and longitude coordinates. This eliminates dependency on external paid map APIs for distance heuristics.

Spherical Haversine Mathematical Derivation:

Given two geographic points $P_1(\phi_1, \lambda_1)$ and $P_2(\phi_2, \lambda_2)$, where ϕ is latitude in radians and λ is longitude in radians:

```
Δϕ = (lat2 − lat1) * π / 180
Δλ = (lon2 − lon1) * π / 180
a  = sin²(Δϕ / 2) + cos(ϕ1) * cos(ϕ2) * sin²(Δλ / 2)
c  = 2 * atan2(√a, √(1 − a))
d  = R * c  (where Earth mean radius R = 6,371,000 meters)
```

C# Optimization: The backend computes `Distance(lat1, lon1, lat2, lon2)` in pure double-precision floating-point arithmetic. LINQ `OrderBy(a ⇒ Distance( … )).FirstOrDefault()` executes within microseconds over in-memory or cached fleets.

3.8 Module 8: Security, Authentication & Role-Based Access Control

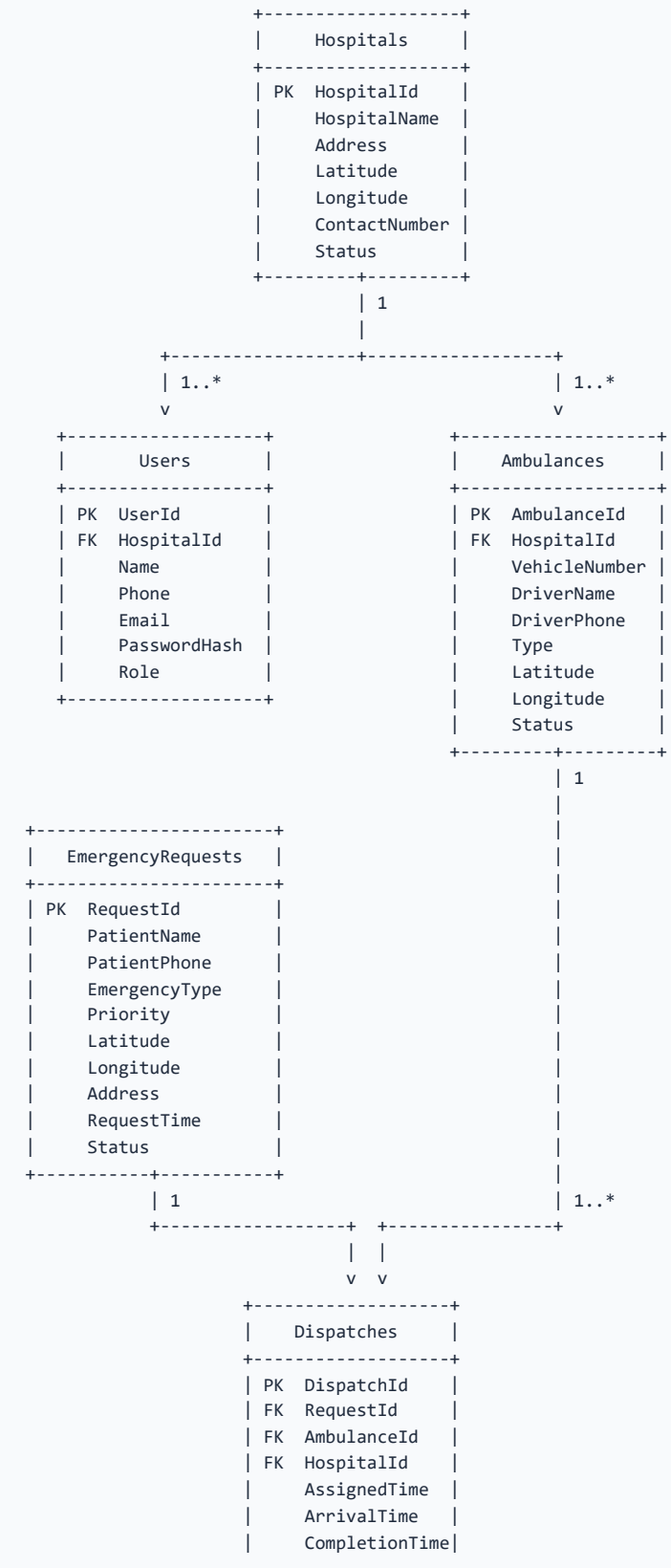
Security & Identity

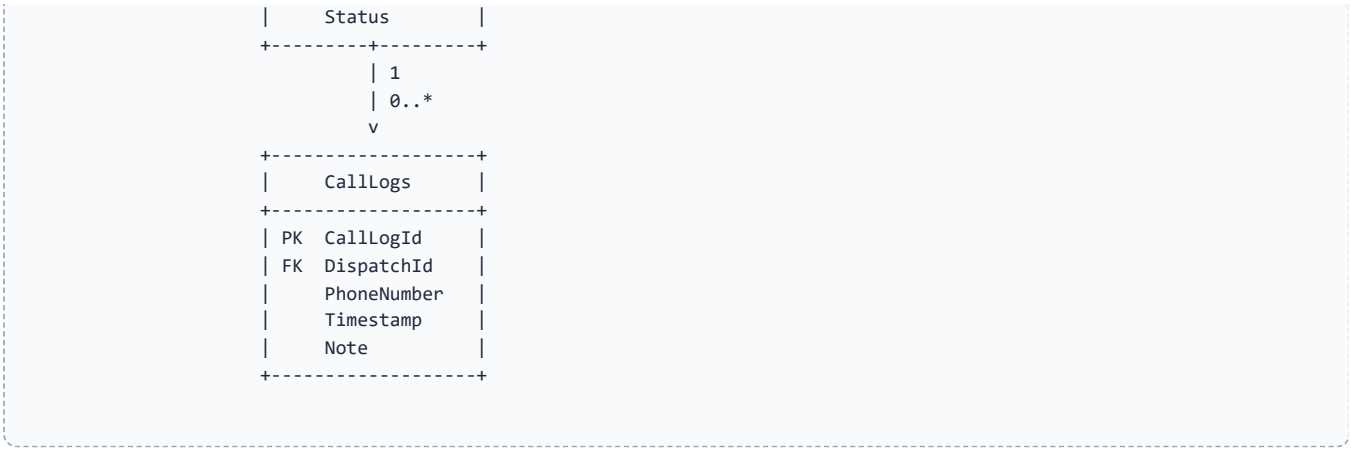
Component Files: `backend/Services/AuthUtils.cs`, `frontend/src/auth.js`, `backend/Program.cs`.

ROLE	AUTHENTICATION METHOD	CLAIMS EMBEDDED IN JWT	AUTHORIZATION SCOPE
Admin	Email / Username + Password (PBKDF2 Hash)	`Role: "Admin"`, `userId`, `Name`	Unrestricted access to all endpoints, fleet CRUD, manual dispatch, debug dump.
Hospital	Hospital Contact Phone + Hospital Name	`Role: "Hospital"`, `hospitalId`, `Name`	Scoped strictly to ambulances and pending requests within their own `HospitalId`.
Driver	Driver Phone + Vehicle Registration Number	`Role: "Driver"`, `ambulanceId`, `hospitalId`	Restricted to mutating their assigned dispatch state and pushing vehicle GPS coordinates.
Citizen / Public	No Login (Anonymous Access)	None	Submit emergency requests, check single request status, read nearest fleet stats.

4. Database Design, Schema & Relational Dictionary

The database architecture is designed with high normalization to eliminate redundancy while preserving referential integrity across the medical dispatch lifecycle.





Data Dictionary & Entity Attribute Specifications

TABLE	FIELD NAME	DATA TYPE	CONSTRAINTS	DESCRIPTION
Users	`UserId`	INT	PK, IDENTITY	Unique user identity identifier.
	`HospitalId`	INT	NULL, FK	Associated hospital for hospital operators.
	`Name`	NVARCHAR(200)	NOT NULL	Full user or administrator name.
	`Phone`	NVARCHAR(50)	NULL	Contact telephone number.
	`Email`	NVARCHAR(200)	NULL	Unique login email identifier.
	`PasswordHash`	NVARCHAR(500)	NOT NULL	Salted PBKDF2/HMACSHA256 password hash.
	`Role`	NVARCHAR(50)	NOT NULL	Authorization role (Admin, Hospital, Driver).
Hospitals	`HospitalId`	INT	PK, IDENTITY	Unique hospital identifier.
	`HospitalName`	NVARCHAR(200)	NOT NULL	Official facility or trauma center name.
	`Address`	NVARCHAR(500)	NULL	Physical civic street address.
	`Latitude`	FLOAT	NOT NULL	WGS84 GPS latitude (-90 to +90).
	`Longitude`	FLOAT	NOT NULL	WGS84 GPS longitude (-180 to +180).
	`ContactNumber`	NVARCHAR(50)	NOT NULL	Direct line for hospital dispatch desk.
	`Status`	NVARCHAR(50)	NOT NULL	Operational status (Open, Closed, Diverting).
Ambulances	`AmbulanceId`	INT	PK, IDENTITY	Unique ambulance fleet identifier.
	`HospitalId`	INT	NULL, FK	Managing hospital facility (ON DELETE SET NULL).
	`VehicleNumber`	NVARCHAR(50)	NOT NULL	Official license registration number plate.

TABLE	FIELD NAME	DATA TYPE	CONSTRAINTS	DESCRIPTION
	`DriverName`	NVARCHAR(200)	NOT NULL	Assigned primary paramedic or driver name.
	`DriverPhone`	NVARCHAR(50)	NOT NULL	Driver contact number used for mobile login.
	`Type`	NVARCHAR(50)	NOT NULL	Vehicle capability (Basic, ALS, Cardiac, Neonatal).
	`Latitude`	FLOAT	NOT NULL	Current live GPS latitude.
	`Longitude`	FLOAT	NOT NULL	Current live GPS longitude.
	`Status`	NVARCHAR(50)	NOT NULL	State: Available, Assigned, Maintenance.
EmergencyRequests	`RequestId`	INT	PK, IDENTITY	Unique emergency distress case number.
	`PatientName`	NVARCHAR(200)	NOT NULL	Name of the victim or reporting citizen.
	`PatientPhone`	NVARCHAR(50)	NOT NULL	Callback number for incoming verification.
	`EmergencyType`	NVARCHAR(100)	NOT NULL	Medical nature (Accident, Cardiac, Trauma, etc.).
	`Priority`	NVARCHAR(50)	NOT NULL	Triage priority (Critical, Urgent, Normal).
	`Latitude`	FLOAT	NOT NULL	Patient pickup GPS latitude.
	`Longitude`	FLOAT	NOT NULL	Patient pickup GPS longitude.
	`Address`	NVARCHAR(500)	NULL	Civic address or nearby landmark notes.
	`RequestTime`	DATETIME2	NOT NULL	UTC timestamp when emergency was initiated.
	`Status`	NVARCHAR(50)	NOT NULL	Status: Pending, Accepted by Hospital, Completed.
Dispatches	`DispatchId`	INT	PK, IDENTITY	Unique dispatch operation record ID.
	`RequestId`	INT	FK	Foreign reference to EmergencyRequests.
	`AmbulanceId`	INT	FK	Foreign reference to Ambulances.
	`HospitalId`	INT	FK	Foreign reference to Hospitals.
	`AssignedTime`	DATETIME2	NOT NULL	UTC timestamp when unit was dispatched.
	`ArrivalTime`	DATETIME2	NULL	UTC timestamp when unit reached scene.
	`CompletionTime`	DATETIME2	NULL	UTC timestamp when patient reached ER.
	`Status`	NVARCHAR(50)	NOT NULL	Assigned, En Route, Arrived, Completed.

5. Complete REST API Specification & Endpoint Directory

The backend exposes a lightweight, high-performance REST API developed in ASP.NET Core Minimal APIs. All endpoints return standard JSON and adhere to conventional HTTP status codes.

METHOD	ENDPOINT URI	AUTHORIZATION	PAYLOAD / PARAMETERS	DESCRIPTION
GET	`/api/health`	Public	None	System health heartbeat check.
POST	`/api/auth/login`	Public	`{ email, password }`	Admin authentication generating JWT bearer token.
POST	`/api/auth/driver-login`	Public	`{ phone, vehicleNumber }`	Driver authentication generating JWT token with ambulance claim.
POST	`/api/auth/hospital-login`	Public	`{ phone, hospitalName }`	Hospital authentication generating JWT token with hospital claim.
GET	`/api/emergency`	Public / User	None	List all emergency requests in the system.
GET	`/api/emergency/{id}`	Public / User	`id` (URL param)	Retrieve single emergency request by ID.
POST	`/api/emergency`	Public	`{ patientName, patientPhone, emergencyType, priority, latitude, longitude, address }`	Submit new citizen emergency distress request.
GET	`/api/emergency/available-for-hospital/{hospitalId}`	Public / Hospital	`hospitalId` (URL param)	List pending emergencies within this hospital's catchment area.

METHOD	ENDPOINT URI	AUTHORIZATION	PAYLOAD / PARAMETERS	DESCRIPTION
POST	`/api/emergency/{id}/accept`	HOSPITAL	`hospitalId` (Query param)	Accepts emergency, auto-assigns nearest ambulance.
GET	`/api/ambulances`	Public / Admin	None	List all registered ambulances.
GET	`/api/ambulances/{id}`	Public / User	`id` (URL param)	Get detailed ambulance records by ID.
GET	`/api/ambulances/nearest`	Public	`?lat=X&lng=Y`	Calculates and returns nearest available ambulance via Haversine.
GET	`/api/hospitals/{id}/ambulances`	HOSPITAL / ADMIN	`id` (URL param)	Retrieve fleet assigned to specific hospital.
POST	`/api/ambulances`	HOSPITAL / ADMIN	`{ hospitalId, vehicleNumber, driverName, driverPhone, type, latitude, longitude, status }`	Register a new ambulance to a hospital fleet.
PUT	`/api/ambulances/{id}`	HOSPITAL / ADMIN	`Ambulance JSON object`	Update ambulance specifications or driver info.
PUT	`/api/ambulances/{id}/location`	DRIVER	`?latitude=X&longitude=Y`	Live driver telemetry endpoint pushing vehicle GPS.
DELETE	`/api/ambulances/{id}`	HOSPITAL / ADMIN	`id` (URL param)	Decommission ambulance (prohibited if active dispatch exists).
GET	`/api/hospitals`	Public	None	List all registered hospitals.

METHOD	ENDPOINT URI	AUTHORIZATION	PAYLOAD / PARAMETERS	DESCRIPTION
GET	`/api/hospitals/nearest`	Public	`?lat=X&lng=Y`	Find nearest hospital facility to coordinates.
POST	`/api/hospitals`	ADMIN	`{ hospitalName, address, latitude, longitude, contactNumber, status }`	Register a new hospital in the directory.
GET	`/api/dispatch`	ADMIN	None	List all active and past dispatch missions.
POST	`/api/dispatch`	ADMIN	`{ requestId, ambulanceId, hospitalId }`	Manual administrative dispatch creation.
PUT	`/api/dispatch/{id}`	DRIVER / HOSPITAL / ADMIN	`{ status, arrivalTime, completionTime }`	Update dispatch state (En Route, Arrived, Completed).
GET	`/api/dispatch/request/{requestId}`	Public	`requestId` (URL param)	Get dispatch assignment linked to an emergency request.
GET	`/api/dispatch/ambulance/{id}`	DRIVER / ADMIN	`id` (AmbulanceId)	List dispatches assigned to specific vehicle.
GET	`/api/dispatch/hospital/{id}`	HOSPITAL / ADMIN	`id` (HospitalId)	List dispatches originating from specific hospital.
GET	`/api/admin/summary`	ADMIN	None	Aggregated KPI metrics for dashboard reporting.
GET	`/api/debug/dump`	ADMIN	None	Full relational table export for monitoring and debugging.

6. Algorithmic Formulations & Mathematical Modeling

The dispatch intelligence hinges on two algorithmic pipelines: **Geodesic Proximity Optimization** and **Hospital Catchment Partitioning**.

6.1 Great-Circle Geodesic Distance (Haversine)

Standard Euclidean distance ($\sqrt{\Delta x^2 + \Delta y^2}$) produces severe spherical distortion over geographical coordinates. The Haversine formulation calculates great-circle distances over Earth's spherical geoid:

Let $R = 6371 \times 10^3\text{ meters}}$ (mean radius of Earth). For coordinates $(\text{lat}_1, \text{lon}_1)$ and $(\text{lat}_2, \text{lon}_2)$ converted to radians:

$$\Delta\phi = (\text{lat}_2 - \text{lat}_1) \cdot \frac{\pi}{180}, \quad \Delta\lambda = (\text{lon}_2 - \text{lon}_1) \cdot \frac{\pi}{180}$$ $$a = \sin^2\left(\frac{\Delta\phi}{2}\right) + \cos(\phi_1)\cos(\phi_2)\sin^2\left(\frac{\Delta\lambda}{2}\right)$$ $$c = 2 \cdot \text{atan2}\left(\sqrt{a}, \sqrt{1-a}\right)$$ $$d = R \cdot c$$

6.2 Hospital Catchment Partitioning Algorithm

To prevent multiple hospitals from simultaneously deploying ambulances to the same victim, the system executes an automated spatial partition when querying pending cases:

```
Algorithm CatchmentPartition(Hospital H_current, List<Hospital> OpenHospitals, List<EmergencyRequest> PendingRequests):
  Initialize AvailableList = []

  For each Request R in PendingRequests:
      NearestHospital = null
      MinDistance = +Infinity

      For each Hospital H in OpenHospitals:
          Dist = Haversine(R.Latitude, R.Longitude, H.Latitude, H.Longitude)
          If Dist < MinDistance Then:
              MinDistance = Dist
              NearestHospital = H

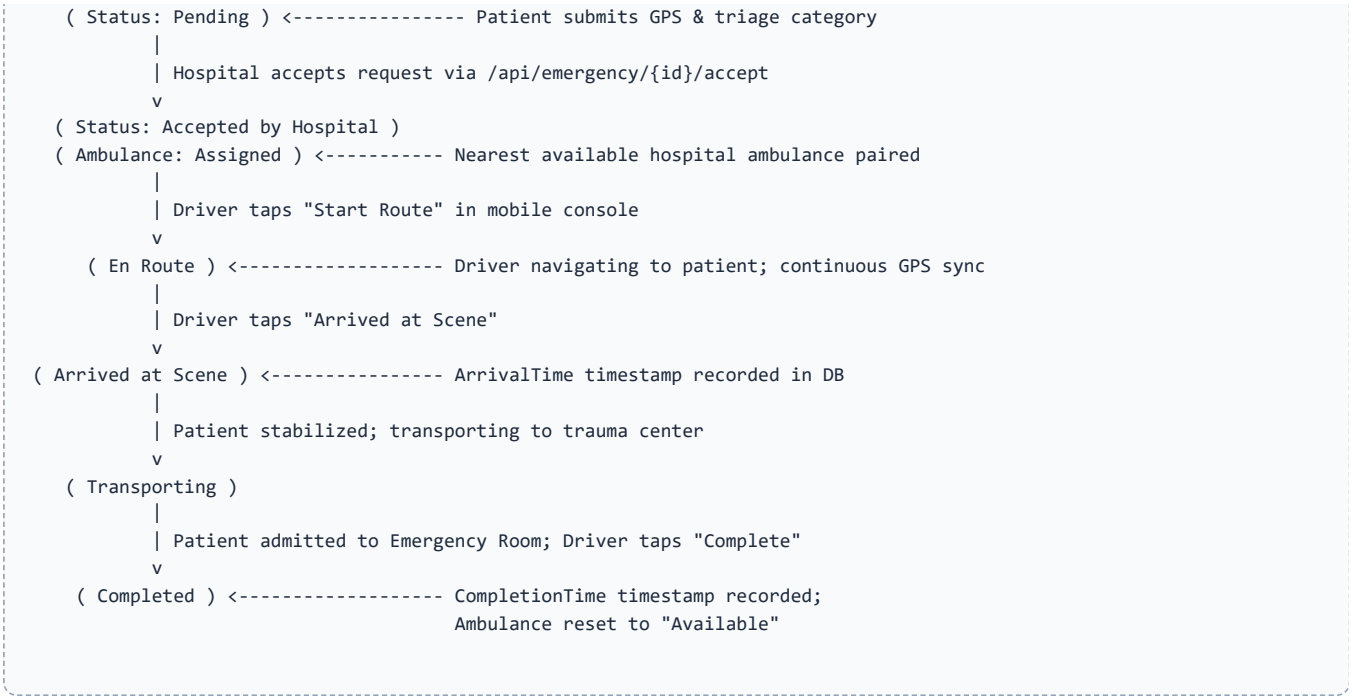
      // Only include if THIS hospital is the absolute closest facility to the patient
      If NearestHospital.HospitalId == H_current.HospitalId Then:
          AvailableList.Add(R)

  Sort AvailableList ascending by Haversine(H_current, R)
  Return AvailableList
```

7. Dispatch State Machine & Operational Lifecycle

An emergency incident progresses through a strict, deterministic finite state machine (FSM). State changes are validated across relational entities to guarantee operational consistency.

```
[ CITIZEN DISTRESS ]
         |
         v
```


(Status: Pending) <---------- Patient submits GPS & triage category
| Hospital accepts request via /api/emergency/{id}/accept
v
(Status: Accepted by Hospital)
(Ambulance: Assigned) <---------- Nearest available hospital ambulance paired
| Driver taps "Start Route" in mobile console
v
(En Route) <---------- Driver navigating to patient; continuous GPS sync
| Driver taps "Arrived at Scene"
v
(Arrived at Scene) <---------- ArrivalTime timestamp recorded in DB
| Patient stabilized; transporting to trauma center
v
(Transporting)
| Patient admitted to Emergency Room; Driver taps "Complete"
v
(Completed) <---------- CompletionTime timestamp recorded;
Ambulance reset to "Available"

8. Deployment, Setup & Smoke Verification Guide

The system is pre-configured to run out of the box in development environments with zero setup friction, featuring automatic database fallback and dual frontend runtimes.

8.1 Backend Configuration (.NET Minimal API)

1. Open a PowerShell terminal and navigate to the backend directory:

```
cd d:\new_project\backend
dotnet restore
dotnet build
```

2. Run database migrations (optional; if SQL Server is absent, the backend automatically activates the In-Memory database provider):

```
dotnet ef database update
```

3. Start the backend server (binds to `http://0.0.0.0:5000` to allow mobile LAN devices to connect):

```
dotnet run
```

4. Verify Swagger OpenAPI documentation at: `http://localhost:5000/swagger`

8.2 Frontend Configuration (React / Expo)

1. Open a second PowerShell terminal in the frontend directory:

```
cd d:\new_project\frontend
npm install
npm start
```

2. Press `w` in the Expo terminal to launch the **Desktop Command Center Web Console** in your default web browser (at `http://localhost:8081`).

3. For mobile testing on physical Android or iOS phones, open **Expo Go** and scan the terminal QR code, or use the camera QR scanner inside the app to pair with your computer's IP address.

8.3 Automated Smoke Verification Script

The repository includes an end-to-end automated PowerShell integration smoke test located at `tests/smoke_test.ps1`. To execute full test verification:

```
pwsh -File d:\new_project\tests\smoke_test.ps1
```

> The script validates: Admin authentication → Hospital creation → Ambulance creation → Emergency submission → Haversine nearest search → Dispatch pairing → Call log recording → Dispatch verification.